Online Training Program
- ASP.NET Core Web API Online Training

ASP.NET WEB API – Basics
- Introduction to ASP.NET Web API
- Creating ASP.NET Web API Application
- How to add Swagger in Web API
- How to use Fiddler to test Web API
- How to use POSTMAN to test Web API
- ASP.NET Web API using SQL Server
- Content Negotiation in Web API
- Media Type Formatter in Web API
- How to Implement GET Method in Web API
- How to Implement POST Method in Web API
- How to Implement PUT Method in Web API
- How to Implement DELETE Method in Web API
- Custom Method Names in Web API
- Parameter Binding in Web API
- Consuming Web API Service From jQuery
- Calling Web API Service in a Cross Domain Using jQuery AJAX
- Cross-Origin Resource Sharing in WEB API

WEB API Routing and Attribute Routing
- ASP.NET Web API Routing
- Routing Variations in WEB API
- Routing and Action Selection in Web API
- ASP.NET Web API Attribute Routing
- Optional Parameters in Web API Attribute Routing
- **Route Prefix in Web API**
- Web API Attribute Routing Route Constraints
- Route Names and Route Orders in Attribute Routing

ASP.NET WEB API – Security
- Authentication and Authorization in ASP.NET Web API
- ASP.NET Web API Basic Authentication
- Role-Based Basic Authentication in Web API
- Web API Service with Basic Authentication
- HTTP Message Handlers in WEB API
- Basic Authentication Using Message Handler in Web API
- HTTP Client Message Handler in Web API
- Token Based Authentication in ASP.NET Web API
- Client Validation Using Basic Authentication in Web API
- Refresh Token in Web API
- Consume Refresh Token in C# Client
- HMAC Authentication in Web API

Web API Versioning
- Web API Versioning Using URI
- Web API Versioning using Query string Parameter
- Web API Versioning Using Custom Header
- Web API Versioning Using Accept Header
- Web API Versioning Using Custom Media Types

Advanced Concepts
- Enable SSL in Visual Studio Development Server
- Enable HTTPS in Web API

Popular ASP.NET Web API Books
- Most Recommended ASP.NET Web API Books

Route Prefix in Web API

Back to: ASP.NET Web API Tutorials For Beginners and Professionals

Attribute Routing Route Prefix Web API

In this article, I am going to discuss the **Attribute Routing Route Prefix in Web API** with some examples. We are going to work with the same example that we started in **Web API Attribute Routing** article and continue in **Optional Parameters in Web API Attribute Routing** article of this Web API article series. At the end of this article, you will understand What is Route Prefix in Web API and when and how to use Web API Route Prefix with an example.

ASP.NET Web API Attribute Routing Route Prefix

Let's understand the use of Web API Attribute Routing Route Prefix with one example. Let's modify the StudentController class as shown below.

```
using System.Collections.Generic;
using System.Linq;
using System.Web.Http;

namespace AttributeRoutingInWEBAPI.Controllers
{
    public class StudentsController : ApiController
    {
        static List<Student> students = new List<Student>()
        {
            new Student() { Id = 1, Name = "Pranaya" },
            new Student() { Id = 2, Name = "Priyanka" },
            new Student() { Id = 3, Name = "Anurag" },
            new Student() { Id = 4, Name = "Sambit" }
        };
        [HttpGet]
        [Route("students")]
        public IEnumerable<Student> GetAllStudents()
        {
            return students;
        }

        [HttpGet]
        [Route("students/{studentID}")]
        public Student GetStudentById(int studentID)
        {
            Student studentDetails = students.FirstOrDefault(s => s.ID == studentID);
            return studentDetails;
        }
        [HttpGet]
        [Route("students/{studentID}/courses")]
        public IEnumerable<string> GetStudentCourses(int studentID)
        {
            List<string> CourseList = new List<string>();
            if (studentID == 1)
                CourseList = new List<string>() { "ASP.NET", "C#.NET", "SQL Server" };
            else if (studentID == 2)
                CourseList = new List<string>() { "ASP.NET MVC", "C#.NET", "ADO.NET" };
            else if (studentID == 3)
                CourseList = new List<string>() { "ASP.NET WEB API", "C#.NET", "Entity
Framework" };
            else
                CourseList = new List<string>() { "Bootstrap", "jQuery", "AngularJs" };
            return CourseList;
        }
    }
}
```

As you can see from the above example, we are using the route attributes at the action level to define the routes, and furthermore, all the routes in the **StudentsController** start with the same prefix – students that mean students is the common prefix for all the routes available in the Student Controller.

Here, you can set the common prefix "**students**" for the entire Student Controller by using the **[RoutePrefix]** attribute as shown below at the controller level.

```
using System.Collections.Generic;
using System.Linq;
using System.Web.Http;

namespace AttributeRoutingInWEBAPI.Controllers
{
    [RoutePrefix("students")]
    public class StudentsController : ApiController
    {
        static List<Student> students = new List<Student>()
        {
            new Student() { Id = 1, Name = "Pranaya" },
            new Student() { Id = 2, Name = "Priyanka" },
            new Student() { Id = 3, Name = "Anurag" },
            new Student() { Id = 4, Name = "Sambit" }
        };
        [HttpGet]
        [Route]
        //This will be translated to /students
        public IEnumerable<Student> GetAllStudents()
        {
            return students;
        }
        [HttpGet]
        [Route("{studentID}")]
        //This will be translated to /students/2
        public Student GetStudentById(int studentID)
        {
            Student studentDetails = students.FirstOrDefault(s => s.ID == studentID);
            return studentDetails;
        }
        [HttpGet]
        [Route("{studentID}/courses")]
        //This will be translated to /students/2/course
        public IEnumerable<string> GetStudentCourses(int studentID)
        {
            List<string> CourseList = new List<string>();
            if (studentID == 1)
                CourseList = new List<string>() { "ASP.NET", "C#.NET", "SQL Server" };
            else if (studentID == 2)
                CourseList = new List<string>() { "ASP.NET MVC", "C#.NET", "ADO.NET" };
            else if (studentID == 3)
                CourseList = new List<string>() { "ASP.NET WEB API", "C#.NET", "Entity
Framework" };
            else
                CourseList = new List<string>() { "Bootstrap", "jQuery", "AngularJs" };
            return CourseList;
        }
    }
}
```

The Route Prefix attribute eliminates the need to repeat the common prefix "students" on each and every controller action method. However, sometimes we may need to override the route prefix attribute. Let us understand this with an example

First, add a class file with the name **"Teacher.cs"** within the Models Folder. To do so right-click on the model's folder, and then add a new class file with the name **"Teacher.cs"**. Then Copy and paste the following code in it.

```
namespace AttributeRoutingInWEBAPI.Models
{
    public class Teacher
    {
        public int Id { get; set; }
        public string Name { get; set; }
    }
}
```

Add the below **GetTeachers()** action method within the **"StudentsController"**.

```
public IEnumerable<Teacher> GetTeachers()
{
    List<Teacher> teachers = new List<Teacher>()
    {
        new Teacher() { Id = 1, Name = "Manoj" },
        new Teacher() { Id = 2, Name = "Prakash" },
        new Teacher() { Id = 3, Name = "Manas" }
    };

    return teachers;
}
```

After adding the **GetTeachers()** action in the **"StudentsController"** class, we want **GetTeachers() action** to be mapped to the URI **'tech/teachers'**.

```
[Route("tech/teachers")]
public IEnumerable<Teacher> GetTeachers()
{
    List<Teacher> teachers = new List<Teacher>()
    {
        new Teacher() { Id = 1, Name = "Manoj" },
        new Teacher() { Id = 2, Name = "Prakash" },
        new Teacher() { Id = 3, Name = "Manas" }
    };

    return teachers;
}
```

If we use the **[Route]** attribute on **GetTeachers()** method as shown in the above image and when we navigate to **tech/teachers**, we get the following error.

HTTP Error 404.0 - Not Found

The resource you are looking for has been removed, had its name changed, or is temporarily unavailable.

But if we navigate to **/students/tech/teachers** then we get the output as expected that the list of teachers. This is because the **[RoutePrefix("students")]** attribute on StudentsController. Now the question that comes to our mind is how to override the RoutePrefix attribute used in the StudentsController. To override the RoutePrefix we need to use the **~** (tilde) symbol as shown below.

```
[Route("~/tech/teachers")]
public IEnumerable<Teacher> GetTeachers()
{
    List<Teacher> teachers = new List<Teacher>()
    {
        new Teacher() { Id = 1, Name = "Manoj" },
        new Teacher() { Id = 2, Name = "Prakash" },
        new Teacher() { Id = 3, Name = "Manas" }
    };
    return teachers;
}
```

With the above change, now the GetTeachers() action method is mapped to URI **'/tech/teachers'** as expected.

What is the use of the RoutePrefix attribute?

The RoutePrefix attribute is used to specify the common route prefix at the controller level to eliminate the need to repeat the common route prefix on each and every controller action.

How to override the route prefix?

Use **~** character to override the route prefix.

In the next article, I am going to discuss **Web API Route Constraint** in Attribute Routing. Here, In this article, I try to explain the **Web API Route Prefix** with some examples. I hope you enjoy this article.

DOT NET TUTORIALS
Dot Net Tutorials
About the Author: *Pranaya Rout*
Pranaya Rout has published more than 3,000 articles in his 11-year career. Pranaya Rout has very good experience with Microsoft Technologies, including C#, VB, ASP.NET MVC, ASP.NET Web API, EF, EF Core, ADO.NET, LINQ, SQL Server, MYSQL, Oracle, ASP.NET Core, Cloud Computing, Microservices, Design Patterns and still learning new technologies.

4 thoughts on "Route Prefix in Web API"

STEVE NGUYEN
SEPTEMBER 30, 2019 AT 5:40 AM

In the code below i can't sent a request to it. So i change a param from "id" to "studentID" and it work.
[HttpGet]
[Route("{studentID}/courses")]
//This will be translated to /students/2/course
public IEnumerable GetStudentCourses(int id)
{
List CourseList = new List();
if (id == 1)
CourseList = new List() { "ASP.NET", "C#.NET", "SQL Server" };
else if (id == 2)
CourseList = new List() { "ASP.NET MVC", "C#.NET", "ADO.NET" };
else if (id == 3)
CourseList = new List() { "ASP.NET WEB API", "C#.NET", "Entity Framework" };
else
CourseList = new List() { "Bootstrap", "jQuery", "AngularJs" };
return CourseList;
}

Reply

DOT NET TUTORIALS
OCTOBER 5, 2019 AT 7:33 PM

Thank you for identifying the issue. We have corrected this.

Reply

PAUL
FEBRUARY 27, 2021 AT 2:04 AM

In the previous article, the route decorations all start with "api/students". In this article, they all just start with "students", either directly in the route decorations or as the route prefix. I cannot make them work without the string "api/" preceding "students". Was there some other change that needed to be made to something?

Reply

GIGER
DECEMBER 16, 2021 AT 1:02 PM

it can work out,the string "api" can be omit in the attribute route, it only necessary in convention-based route. When you choose use attribute route,you just need to add controller name and parameters.

Reply

Leave a Reply

Your email address will not be published. Required fields are marked *

Comment *

Name* | Email* | Website

Post Comment

© Dot Net Tutorials | Website Design by Sunrise Pixel